nanoGPT Scaling Laws on Colab

Five GPT models (0.10M to 10.65M non-embedding parameters) trained from scratch on TinyStories, on a Tesla T4 (15 GB) in Google Colab.

1. Key results

Metric	Value
Fitted size exponent α in L(N) = E + A·N^-α	0.235
Fitted irreducible loss E	1.26 (nats/token)
Compute-frontier slope, L ~ C^b	b = -0.089
Best / worst final validation loss	1.696 (s4) / 2.549 (s0)
Total T4 training time	≈ 2.3 hours (137.5 min)

Bigger models reached lower loss on the same 100M training tokens, and the improvement follows a smooth power law. The fitted α sits between the Kaplan et al. (≈0.076) and Chinchilla (≈0.34) reference values.

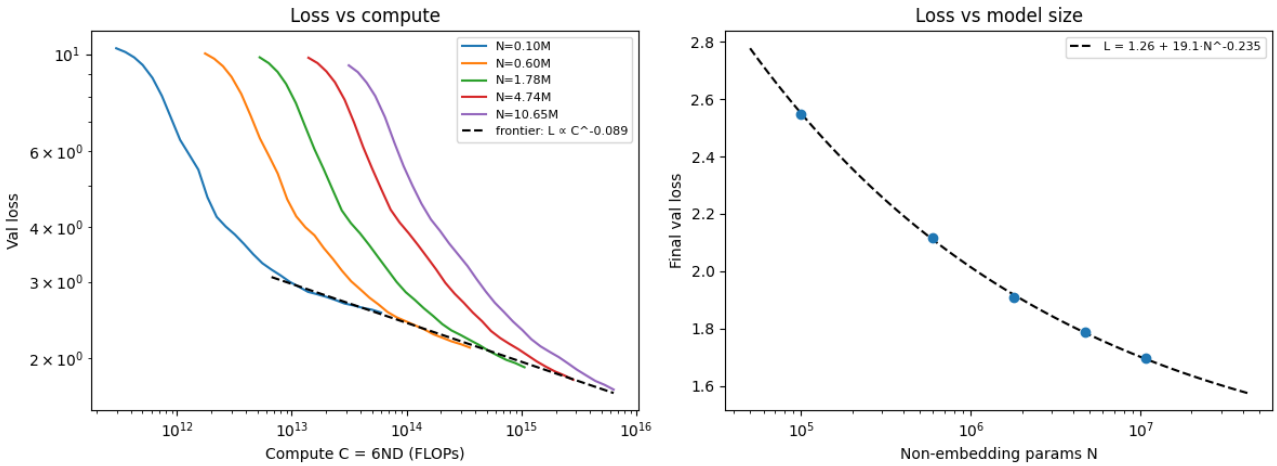
2. Experimental setup

Model: GPT-2-style decoder (pre-LayerNorm, GELU MLP, tied embeddings, flash attention), context 256, GPT-2 vocabulary padded to 50,304.
Data: TinyStories tokenized with tiktoken (GPT-2 BPE). Each model sees 100M training tokens (6,103 steps × 64 × 256).
Optimizer: AdamW (β = 0.9, 0.95; weight decay 0.1), 5% linear warm-up then cosine decay to 10% of peak LR, gradient clip 1.0.
Precision: fp16 autocast with GradScaler (T4 has no bf16).
Evaluation: 30 log-spaced validation points per run, 20 batches each. Compute is estimated as C = 6·N·D.
Seed: one run per model (seed 0).

3. Experiment grid and final losses

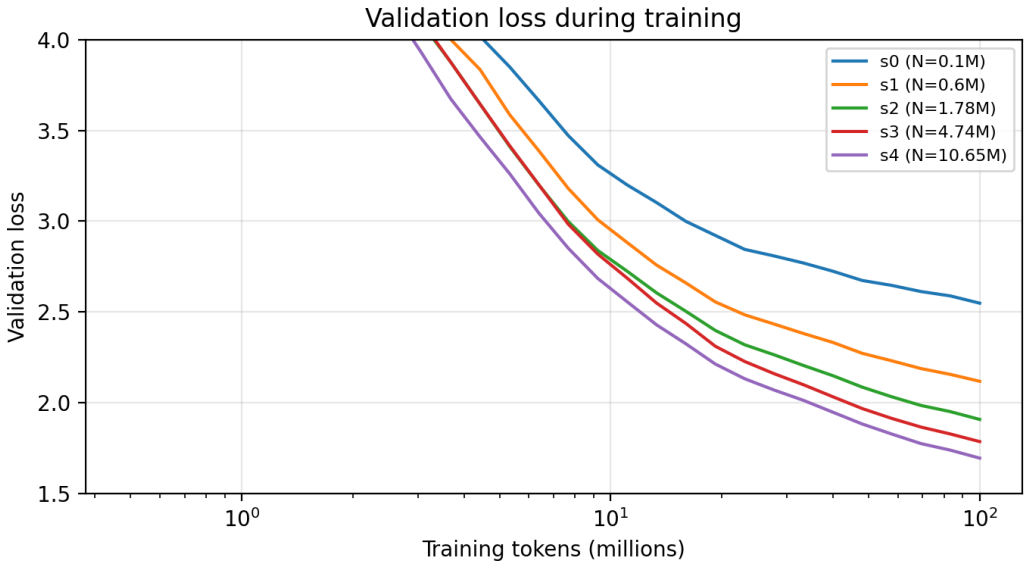
Name	Layers	d_model	Heads	N (M)	Peak LR	Final val loss	T4 time (min)
s0	2	64	2	0.10	3e-3	2.549	20.5
s1	3	128	4	0.60	2e-3	2.119	22.2
s2	4	192	4	1.78	1.5e-3	1.909	25.8
s3	6	256	8	4.74	1e-3	1.787	30.4
s4	6	384	8	10.65	8e-4	1.696	38.6

4. Scaling-law fit



Left: validation loss vs compute, one curve per model; the dashed line is a power-law fit to the lower envelope (compute frontier). Right: final loss vs model size with the fit $L = 1.26 + 19.1 \cdot N^{-0.235}$.

5. Training curves



Every model drops quickly in the first ~10M tokens, then improves slowly. Larger models learn faster per token and finish lower.

6. How to read these results

Loss is the average surprise (cross-entropy, in nats) the model has when predicting the next token. Lower is better. A random guesser over 50k tokens scores about 10.8; the models here reach 1.7 to 2.5.

Scaling law. Loss falls along a smooth curve as the model grows. This makes it possible to predict how a bigger model will behave from small ones.

α = 0.235 measures how fast loss falls with size. Each 10× increase in parameters cuts the reducible part of the loss (L - E) by a factor of $10^{0.235} \approx 1.7$.

E = 1.26 is the loss an infinitely large model would still have, because language contains randomness no model can predict.

Compute frontier. Each curve in the left plot is one model. The best loss for a given compute budget is the lower edge of all curves: small models win at small budgets, large models at large budgets.

7. Limitations

• Only five models and one seed. A 3-parameter fit to 5 points is fragile; E and α are strongly coupled.
• All models train on the same 100M tokens, so the largest are likely under-trained and the fit is not compute-optimal (Chinchilla-style).
• Learning rates were set by hand per size, not tuned.
• The plain log-log slope between the two largest models (s3 to s4) is only ≈ 0.065, below the fitted α, because E is not removed. Across all five models it is ≈ 0.087.
• TinyStories is simple synthetic text; exponents may differ on web-scale data.
• Reference exponents (Kaplan et al. 2020, arXiv 2001.08361; Hoffmann et al. 2022, arXiv 2203.15556) are approximate.